



Reporte de proyecto

Simulador de Sistema Operativo

Equipo:

Lalalito lalala

Integrantes:

Padilla Godínez Eduardo Uziel

Mejorada Ordóñez Héctor Vicente

Camacho Guzman Emilio

Salazar Galván Adolfo

Profesor:

Ariel adara Mercado Martínez

Materia:

Estructura de Datos y Algoritmos

Introducción:

El presente proyecto busca la creación de un simulador de sistema operativo, que gestione procesos y bloques de memoria, generados por python y simulados, respectivamente, para poder así implementar los aprendizajes obtenidos en clase, apoyados por el uso de IA, para al final generar una tabla que relacione procesos/tiempo, para poder así ver cuál es el más adecuado.

Marco Teórico:

Conceptos necesarios para entender el proyecto.

- Sistema operativo.
- Gestión de memoria.
- Fragmentación.
- Coalescencia.
- First Fit.
- Best Fit.
- Worst Fit.
- Listas ligadas.
- Colas y pilas.

Descripción de campos:

Campo	Descripción
pid	Identificador único del proceso
burst_time	Tiempo total requerido de CPU
remaining_time	Tiempo restante de ejecución
priority	Prioridad del proceso
memory_required	Memoria solicitada
state	Estado actual del proceso

Se decidió que todos investigaran en profundidad la estructura del proyecto y tener un primer acercamiento a la forma de abordarlo, durante la investigación se fueron comprendiendo los temas y cómo se enlazaban unos con otros, para después un integrante generar un canva y confirmar que todos hubieran comprendido lo mismo, exponiendo los hallazgos y obteniendo retroalimentación o debatiendo sobre la información encontrada.

De esta manera nos aseguramos de estar en sintonía de lo que estábamos trabajando.

Decidimos seguir el orden de trabajo que nos sugirió la inteligencia artificial, esta consistió en seguir pasos y confirmar que existieran carpetas o archivos.

1. **Estructuras de Datos y Algoritmos:**

Se buscó la implementación de estructuras de datos (stack, Queue, C Queue, LL, LDL).

2. **Simulación de procesos:**

Confirmamos que el modelado de procesos tuviera lo necesario para poder funcionar (PID, Tiempo de Llegada, Tiempo de ejecución requerido, Tiempo restante, Memoria necesaria)(así mismo se definieron estados básicos de ejecución, READY, RUNNING, FINISHED, WAITING)

3. **Memory Manager:**

Se implementó el administrador de memoria simulada, a través de bloques de memoria capaz de representar espacios de memoria libres y ocupados

- a. **Inicio:** se creó un administrador de memoria que inicia con un único bloque libre, que es toda la memoria disponible
- b. **First Fit:** Algoritmo que asigna memoria utilizando el primer bloque libre que sea suficientemente grande para el tamaño del proceso.
- c. **Best Fit:** Algoritmo que se encarga de seleccionar el bloque libre más pequeño capaz de alojar el proceso.
- d. **Worst Fit:** Algoritmo que se encarga de seleccionar el bloque libre más grande.
- e. **Liberación de memoria:** Se añadió la capacidad de liberar memoria cuando un proceso termina su función
- f. **Coalescencia:** Une bloques adyacentes, al parecer se utiliza para reducir la fragmentación externa

- g. **Compactación:** Se desarrolló un algoritmo de compactación para reorganizar bloques de memoria y concentrar espacio libre.

4. Scheduler:

confirmando que no hubiese errores, pasamos a el planificador de procesos, en base a él se construyó la gráfica final, haciendo una comparación en qué tipo de planificador de procesos es más adecuado ocupar, o el peor.

- a. **FIFO:** *“primero en entrar primero en salir”*, conforme llegaban los procesos era el orden de salida, el concepto es relativamente sencillo.
- b. **SJF:** *“primero el trabajo más corto”*, se fija en el proceso con menor tiempo de ejecución.
- c. **RR:** *“proporciona equidad”*, por medio de *quantums* distribuye el tiempo del cpu de manera equitativa, haciendo el proceso cíclico.

5. Unión global:

Se realizó un primer intento de coordinar todo lo que llevábamos hasta el momento, aún sin realizar ninguna prueba.

El programa coordinó las siguientes actividades:

- a. **Creación del administrador de memoria**
- b. **Carga de procesos**
- c. **Selección del scheduler**
- d. **Asignación de memoria**
- e. **Ejecución simulada**
- f. **Liberación de memoria**
- g. **Registro de estadísticas (csv)**

6. Pruebas y validación:

Se corrieron y adecuaron los pruebas para verificar el funcionamiento de cada módulo, a pesar de ser modificadas fueron exitosas en su cometido

Las pruebas incluyeron:

- a. **Operaciones sobre pila**
- b. **Operaciones sobre colas**
- c. **Colas circulares**
- d. **Administración de memoria**
- e. **Planificación de procesos**
- f. **Integración**

7. Benchmarking y Generación de reportes:

Para obtener estadísticas en tablas, datos arrojados y en general, evaluar el desempeño del simulador, se corrieron y adecuaron los scripts de python. Los cuales permitieron exitosamente:

- Generar procesos automáticamente
- Ejecutar múltiples configuraciones experimentales
- Generar tablas comparativas
- Exportar resultados a archivos .png y .csv

Durante la creación del proyecto se agregaron carpetas y archivos para poder facilitar el proceso de ejecución y el objetivo general.

Problemas/Soluciones: Durante la creación del programa surgieron muchos problemas de código, fue allí donde la IA fue de gran ayuda para detectarlos rápido y hacer una sugerencia de cambio, sin embargo, en este apartado serán expuesto los problemas con los que tuvimos que hacer un uso crítico de la IA.

1° Problema

Durante la creación del proyecto hubo confusiones con la IA e inclusive con VSC que tuvimos que solucionar empleando las herramientas que obtuvimos durante el curso; en la unión global tardamos tiempo en unir todo, pues como realizamos diferentes partes en diferente tiempo con IA, algunos puntos no coincide y arrojaban errores muy largos que era estresante leer, por ende se copiaba y pegaba a chat para poder comprender el error y buscar una forma de solucionarlo, todos intentábamos hacerlo y lo empeoramos más, por ende se decidió cada quien hacerlo por separado, sin embargo, después de 2 horas pudimos solucionar todo y obtener el .png de la gráfica, confirmamos que los resultados fuesen coherentes y dimos por terminada el programa.

2° Problema

en la parte de unir los .py con los .c no sucedía nada, tardamos 1 hora en darnos cuenta de que el compañero que estaba compartiendo el código no tenía instalado python y al intentar instalarlo no lo logró por requisitos de su computadora, por ende hicimos push y lo continuamos en code spaces.

3° Problema

Realmente no teníamos idea de cómo compactar fifo, rr y sjf, por ende decidimos hacer un archivo extra, llamado **scheduler.c**, que de esta forma logramos unir todo scheduler de una manera sencilla

4° Problema

Realmente no teníamos idea de cómo compactar fifo, rr y sjf, por ende decidimos hacer un archivo extra, llamado **scheduler.c**, que de esta forma logramos unir todo scheduler de una manera sencilla, siendo este el primer caso de carpetas agregadas, donde además entran:

process_loader.c: para poder leer un archivo de entrada y convertirlo en un proceso para scheduler y el administrador de memoria

benchmark: carpeta para poder obtener datos de salida en .csv con las condiciones indicadas (algoritmo, tamaño y tiempo de ejecución) y el .png para obtener la gráfica

data: para poder obtener los datos que arrojó python de forma .csv (generate_process.py)

Complejidad Big O por cada archivo

n = número de procesos

B = número de bloques de memoria

C = capacidad de memoria

L = número de nodos

Q = número de turnos de Round Robin

K = capacidad de la cola circular

src/algorithms/

backtracking.c

Su complejidad temporal es $O(n*B)$ porque recorre los procesos y por cada uno intenta asignar memoria buscando entre los bloques. Su complejidad espacial es $O(n)$ porque usa recursión y puede acumular llamadas en la pila.

brute_force.c

Su complejidad temporal es $O(B)$ porque recorre los bloques de memoria hasta encontrar un hueco válido. Su complejidad espacial es $O(1)$ porque solo usa variables auxiliares.

divide_conquer.c

Su complejidad temporal a la hora de ordenar procesos con Merge Sort es $O(n \log n)$, y para compactar memoria es $O(B)$. Su complejidad espacial al ordenar es $O(n)$ porque Merge Sort usa arreglos auxiliares, mientras que al compactar es $O(1)$.

dp_bottomup.c

Su complejidad temporal es $O(n*C)$ porque llena una tabla de programación dinámica usando procesos y capacidad de memoria. Su complejidad espacial también es $O(n*C)$ porque guarda esa tabla en memoria.

dp_topdown.c

Su complejidad temporal es $O(n*C)$ porque usa recursión con memoización, calculando cada estado una sola vez. Su complejidad espacial también es $O(n*C)$ porque guarda los resultados en una tabla de memoización.

greedy.c

Su complejidad temporal es $O(B)$ porque los métodos First Fit, Best Fit y Worst Fit recorren los bloques de memoria para encontrar uno adecuado. Su complejidad espacial es $O(1)$ porque no crea estructuras grandes adicionales.

incremental.c

Su complejidad temporal es $O(1)$ ya que solo actualiza datos de un proceso. Su complejidad espacial también es $O(1)$ porque no usa memoria extra relevante.

src/memory/

La complejidad temporal de todos los archivos exceptuando `memory_manager` es de $O(B)$ y la espacial es de $O(1)$ ya que todas recorren todos los bloques.

memory_manager.c

Su complejidad temporal al crear el memory manager es $O(1)$, pero al asignar, liberar o destruir memoria puede ser $O(B)$, porque puede recorrer los bloques. Su complejidad espacial es $O(B)$ porque el administrador guarda una lista de bloques de memoria.

src/scheduler/

fifo.c

Su complejidad temporal es $O(n)$ ya que busca el siguiente proceso disponible recorriendo procesos. Su complejidad espacial es $O(n)$ porque el scheduler guarda los procesos en memoria.

sjf.c

Su complejidad temporal es $O(n)$ por selección y $O(n^2)$ en total, ya que va buscando el proceso más corto entre los no terminados. Su complejidad espacial es $O(n)$ porque almacena los procesos.

round_robin.c

Su complejidad temporal es $O(n)$ por turno y $O(Q*n)$ en total, ya que en cada turno puede revisar varios procesos hasta encontrar uno ejecutable. Su complejidad espacial es $O(n)$ porque guarda los procesos en el scheduler.

scheduler.c

Su complejidad temporal es $O(n)$ porque funciones como `scheduler_is_empty` recorren todos los procesos. Agregar procesos tiene complejidad $O(1)$ amortizada. Su complejidad espacial es $O(n)$ porque guarda un arreglo de procesos.

src/

stack.c

Su complejidad temporal es $O(1)$ en operaciones como push, pop y peek, porque trabaja directamente con el tope de la pila. Al destruir la pila es $O(L)$ porque recorre todos los nodos. Su complejidad espacial es $O(L)$ porque guarda L nodos.

queue.c

Su complejidad temporal es $O(1)$ en operaciones como enqueue, dequeue y front, porque tiene apuntadores al inicio y al final. Al destruir la cola es $O(L)$. Su complejidad espacial es $O(L)$ porque guarda L nodos.

circular_queue.c

Su complejidad temporal es $O(1)$ porque insertar, sacar y consultar usan índices circulares. Su complejidad espacial es $O(K)$ porque reserva un arreglo con capacidad K .

linked_list.c

Su complejidad temporal es $O(L)$ en operaciones como buscar, destruir o insertar al final, porque puede recorrer toda la lista. Insertar al inicio es $O(1)$. Su complejidad espacial es $O(L)$ porque guarda L nodos.

doubly_linked_list.c

Su complejidad temporal es $O(1)$ al insertar o quitar en los extremos, porque tiene apuntadores al inicio y al final. Buscar o destruir es $O(L)$ porque recorre la lista. Su complejidad espacial es $O(L)$ porque guarda L nodos.

main.c

Su complejidad temporal depende del scheduler usado. Con FIFO o SJF puede ser $O(n^2)$, y con Round Robin puede ser $O(Q*n)$. Su complejidad espacial es $O(n)$ porque carga y guarda los procesos.

src/utils/

process_loader.c

Su complejidad temporal es $O(n)$ porque lee los procesos del archivo uno por uno. Su complejidad espacial es $O(n)$ porque guarda los procesos en un arreglo dinámico.

tests/

tests/main.c

Su complejidad temporal es $O(1)$ porque ejecuta pruebas con cantidades fijas. Su complejidad espacial también es $O(1)$.

test_circular_queue.c

Su complejidad temporal es $O(1)$ con los datos actuales, pero generalizada puede ser $O(K)$ si se prueban K elementos. Su complejidad espacial es $O(K)$ porque usa una cola circular de capacidad K .

test_memory_manager.c

Su complejidad temporal es $O(B)$ porque prueba asignaciones y liberaciones que recorren bloques de memoria. Su complejidad espacial es $O(B)$ porque el memory manager puede crear varios bloques.

test_queue.c

Su complejidad temporal es $O(1)$ con los datos actuales, pero generalizada sería $O(L)$ si se insertan o eliminan L elementos. Su complejidad espacial es $O(L)$.

test_scheduler.c

Su complejidad temporal depende del scheduler probado. FIFO puede ser $O(n)$, SJF puede llegar a $O(n^2)$, y Round Robin puede ser $O(Q*n)$. Su complejidad espacial es $O(n)$ porque guarda procesos.

test_stack.c

Su complejidad temporal es $O(1)$ con los datos actuales, pero generalizada sería $O(L)$ si se insertan o eliminan L elementos. Su complejidad espacial es $O(L)$.